

Regler les gains PIDF

Points de départ et procédure de réglage pour les gains de vos moteurs de propulsion et d'orientation.

Les moteurs d'orientation/angles/azimut du Swerve ont le bouclage PID (PID wrapping) actif aux points le plus haut et le plus bas (par exemple avant et arrière). Lors du réglage du PID d'angle, il est conseillé de tester avec un translation vers la gauche ou la droite afin que le module ne croise pas le point de bouclage.

L'idée de base

YAGSL lit les valeurs PIDF a partir du fichier [pidfproperties.json](#) situe dans le dossier de votre module sur [config.yagsl.com](#).

Cela a deja ete résumé de la manière la plus simple comme ceci:

P: Si vous n'êtes pas la ou vous voulez être, allez-y

I: Si vous n'êtes pas là où vous voulez être depuis longtemps, allez-y plus vite.

D: Si vous vous rapprochez de la ou vous voulez être, ralentissez

[Video](#) trouvée par 78, citée dans ce post [this CD post](#)

Comment les paramètres PID affectent-ils la dynamique du système?

S-S Error

Les effets de l'augmentation de chacun des paramètres du contrôleur K_p , K_i , and K_d peuvent etre resumes ainsi:

Parametre	Temps de montee	Depassement	Temps d'établissement	Erreur en régime permanent
K_p	Diminue	Augmente	NT	Diminue
K_i	Diminue	Agumente	Augmente	Elimine
K_d	NT	Diminue	Diminue	NT

NT: Pas de tendance define. Changement mineur.
Vous devriez prendre des notes sur ce tableau. Il sera utile dans la suite de la lecon.

Procedure de reglage

Une autre bonne façon de la décrire, [from CTRE](#):

Le réglage manuel suit généralement ce processus:

1. Reglez P, I, and D a zero.

2. Augmentez P jusqu'à ce que la sortie commence à osciller autour de la consigne.
3. Augmentez D autant que possible sans introduire de tremblement dans la réponse.

Pour le moteur de propulsion, réglez également les termes de feedforward (s,v, a voir ci-dessous) avant d'affiner P ; un bon feedforward signifie que P n'a qu'à corriger de petites erreurs.

Guide pas à pas WPILib

WPILib dispose d'une excellente documentation sur le PID, cela vaut donc la peine de la lire directement:

[Basics of PID](#)

[Tuning velocity, like the drive motors!](#)

[Tuning position, like the steering/angle/azimuth motors!](#)

La structure actuelle de pidfproperties.json

Le schema a change dans YAGSL 2026.8.05 — **f** and **iz** (zone integrale) et l'ancien limiteur de sortie ont disparu. Le feedforward est maintenant exprime sous la forme **s/v/a** (**kS/kV/kA**, correspondent au WPILib's **SimpleMotorFeedforward**). Consultez la référence des changements: [schema changes référence si](#) vous migrez manuellement une ancienne configuration.

None

```
{
  "drive": {
    "p": 0.02,
    "i": 0,
    "d": 0,
    "s": 0,
    "v": 0,
    "a": 0
  },
  "angle": {
    "p": 0.01,
    "i": 0,
    "d": 0,
    "s": 0,
    "v": 0,
    "a": 0
  }
}
```

```
}
```

Si vous laissez `v` (kV) de la propulsion a `0`, YAGSL le calcule automatiquement a partir de la vitesse à vide du moteur de propulsion et de votre engrenage, de sorte qu'un feedforward raisonnable existe avant même que vous n'ayez réglé quoi que ce soit à la main.

Vous pouvez également ajouter un fichier `pidfproperties_sim.json` a cote — YAGSL le chargera a la place lors de l'exécution en simulation (`RobotBase.isSimulation()`), vous permettant d'utiliser des gains plus souples en simulation sans modifier la configuration de votre robot de compétition.

Points de depart

Les bonnes valeurs PIDF varient d'un robot à l'autre, mais voici des points de départ raisonnables pour commencer vos iterations, organisés par type de contrôleur de moteur.

SparkMax / SparkFlex

None

```
{
  "drive": {
    "p": 0.0020645,
    "i": 0,
    "d": 0,
    "s": 0,
    "v": 0,
    "a": 0
  },
  "angle": {
    "p": 0.01,
    "i": 0,
    "d": 0,
    "s": 0,
    "v": 0,
    "a": 0
  }
}
```

TalonFX

Lorsque le type de moteur de propulsion pour chaque module est `talonfx_*` (our similaire comme — `krakenx60`, `falcon500`), voici un point de départ raisonnable:

None

```
{
  "drive": {
    "p": 1,
    "i": 0,
    "d": 0,
    "s": 0,
    "v": 0,
    "a": 0
  },
  "angle": {
    "p": 50,
    "i": 0,
    "d": 0.32,
    "s": 0,
    "v": 0,
    "a": 0
  }
}
```

Une fois que vous avez un point de départ raisonnable, utilisez config.yagsl.com pour regénérer `pidfproperties.json` avec vos chiffres mis à jour, ou modifiez directement le fichier à la main. Les deux méthodes fonctionnent, car l'analyseur (parser) ne prend en compte que le JSON final.